复旦大学信息科学与工程学院

《深入理解计算机系统》期末考试试卷

A 卷　共 12 页

课程代码：372.073.1.03　　**考试形式：□开卷　√闭卷**　　2007 年 12 月

（本试卷答卷时间为 120 分钟，答案必须写在试卷上，做在草稿纸上无效）

专业_____学号_____姓名_____成绩_____

题号	一	二	三	四	五	六	总分
得分							

（装订线内不要答题）

Problem 1. (15 points)

Consider the following assembly code for a C for loop:

```
    loop:
    pushl   %ebp
    movl    %esp,%ebp
    movl    8(%ebp),%ecx
    movl    12(%ebp),%edx
    xorl    %eax,%eax
    cmpl    %edx,%ecx
    jle .L4
.L6:
    decl    %ecx
    incl    %edx
    incl    %eax
    cmpl    %edx,%ecx
    jg      .L6
.L4:
    incl    %eax
    movl    %ebp,%esp
    popl    %ebp
    ret
```

Based on the assembly code above, fill in the blanks below in its corresponding C source code. (Note: youmay only use the symbolic variables *x*, *y*, and *result* in your expressions below — *do not use register names.*)

```
int loop(int x, int y)
{
    int   result;
    for (_____________; _____________; result++ ) {
        _____________;
        _____________;
    }
    _____________;
    return result;
}
```

Problem 2. (12 points)

Consider the following incomplete definition of a C struct along with the incomplete code for a function func given below.

```
typedef    struct node {
        _________________ x;
        _________________ y;
        struct node *next;
        struct node *prev;
} node_t;

node_t n;
void func() {
        node_t *m;
        m = _____________________________;
        m->y /= 16;
        return;
}
```

When this C code was compiled on an IA-32 machine running Linux, the following assembly code was generated for function func.

```
func:
pushl     %ebp
movl      n+12,%eax
movl      16(%eax),%eax
movl      %esp,%ebp
movl      %ebp,%esp
shrw      $4,8(%eax)
popl      %ebp
ret
```

Given these code fragments, fill in the blanks in the C code given above. Note that there is a unique answer. The types must be chosen from the following table, assuming the sizes and alignment given.

Type	char	short	unsigned short	int	unsigned int	double
Size(bytes)	1	2	2	4	4	8
Alignment	1	2	2	4	4	8

（装订线内不要答题）

Problem 3. (15 points)

The following problem concerns optimizing a procedure for maximum performance on an Intel Pentium III. Recall the following performance characteristics of the functional units for this machine:

Operation	Latency Time	Issue Time
Integer Add	1	1
Integer Multiply	4	1
Integer Divide	36	36
Floating Point Add	3	1
Floating Point Multiply	5	2
Floating Point Divide	38	38
Load or Store (Cache Hit)	1	1

Consider the following two procedures:

Loop 1

```
int loop1(int *a, int x, int n)
{
    int y = x*x;
    int i;
    for (i = 0; i < n; i++)
        x = y * a[i];
    return x*y;
}
```

Loop 2

```
int loop2(int *a, int x, int n)
{
    int y = x*x;
    int i;
    for (i = 0; i < n; i++)
        x = x * a[i];
    return x*y;
}
```

When compiled with GCC, we obtain the following assembly code for the inner loop:

Loop 1

```
.L21:
    movl    %ecx,%eax
    imull   (%esi,%edx,4),%eax
    incl    %edx
    cmpl    %ebx,%edx
    jl      .L21
```

Loop 2

```
.L27:
    imull   (%esi,%edx,4),%eax
    incl    %edx
    cmpl    %ebx,%edx
    jl      .L27
```

Running on one of the Fish machines, we find that Loop 1 requires 3.0 clock cycles per iteration, while Loop 2 requires 4.0.

A. Explain how it is that Loop 1 is faster than Loop 2, even though it has one more instruction

B. By using the compiler flag -funroll-loops, we can compile the code to use 4-way loop unrolling. This speeds up Loop 1. Explain why.

C. Even with loop unrolling, we find the performance of Loop 2 remains the same. Explain why.

Problem 4. (24 points)

This problem tests your understanding of cache conflict misses. Consider the following matrix transpose routine

```
typedef  int array[2][2];
void     transpose(array dst, array src) {
    int  i, j;
    for (i = 0; i < 2; i++) {
        for (j = 0; j < 2; j++) {
            dst[j][i] = src[i][j];
```


（装订线内不要答题）

```
        }
    }
}
```

running on a hypothetical machine with the following properties:

- *sizeof(int) == 4.*
- The src array starts at address 0 and the dst array starts at address 16 (decimal).
- There is a single L1 cache that is direct mapped and write-allocate, with a block size of 8 bytes.
- Accesses to the src and dst arrays are the only sources of read and write misses, respectively.

A. Suppose the cache has a total size of 16 data bytes (i.e., the block size times the number of sets is 16 bytes) and that the cache is initially empty. Then for each row and col, indicate whether each access to *src[row][col*] and *dst[row][col]* is a hit (*h*) or a miss (*m*). For example, reading *src[0][0]* is a miss and writing *dst[0][0]* is also a miss.

dst array		
	Col 0	Col 1
Row 0	m	
Row 1		

src array		
	Col 0	Col 1
Row 0	m	
Row 1		

B. Repeat part A for a cache with a total size of 32 data bytes.

dst array		
	Col 0	Col 1
Row 0	m	
Row 1		

src array		
	Col 0	Col 1
Row 0	m	
Row 1		

Problem 5. (24 points)

This problem tests your understanding of exceptional control flow in C programs. Assume we are running code on a Unix machine. The following problems all concern the value of the variable counter.

Part I (9 points)

```
int counter = 0;
int main()
{
    int i;
    for (i = 0; i < 2; i ++){
        fork();
        counter ++;
        printf("counter = %d\n", counter);
    }
    printf("counter = %d\n", counter);
    return 0;
}
```

A. How many times would the value of counter be printed? _____

B. What is the value of counter printed in the first line? _____

C. What is the value of counter printed in the last line? _____

Part II (9 points)

```
pid_t pid;
int counter = 0;
void handler1(int sig)
{
    counter ++;
    printf("counter = %d\n", counter);
    fflush(stdout); /* Flushes the printed string to stdout */
    kill(pid, SIGUSR1);
}
```

（装订线内不要答题）

```
void handler2(int sig)
{
    counter += 3;
    printf("counter = %d\n", counter);
    exit(0);
    }

    void main() {
        signal(SIGUSR1, handler1);
        if ((pid = fork( )) == 0) {
            signal(SIGUSR1, handler2);
            kill(getppid( ), SIGUSR1);
            while(1) { };
        }
        else {
            pid_t p; int status;
            if ((p = wait(&status)) > 0) {
                counter += 2;
                printf("counter = %d\n", counter);
            }
        }
    }
```

What is the output of this program?

Part III (6 points)

```
int counter = 0;
void handler(int sig)
{
    counter ++;
}

int main()
{
    int i;
    signal(SIGCHLD, handler);
    for (i = 0; i < 5; i ++){
        if (fork() == 0){
            exit(0);
        }
    }
    /* wait for all children to die */
```

```
        while (wait(NULL) != -1);
        printf("counter = %d\n", counter);
        return 0;
    }
```

A. Does the program output the same value of counter every time we run it? Yes No

B. If the answer to A is Yes, indicate the value of the counter variable. Otherwise, list all possible values of the counter variable.

Answer: counter = _____

（装订线内不要答题）

Problem 6. (22 points)

The following problem concerns the way virtual addresses are translated into physical addresses.

- The memory is byte addressable.
- Memory accesses are to 1-byte words (not 4-byte words).
- Virtual addresses are 16 bits wide.
- Physical addresses are 13 bits wide.
- The page size is 512 bytes.
- The TLB is 8-way set associative with 16 total entries.
- The cache is 2-way set associative, with a 4 byte line size and 16 total lines.

In the following tables, all numbers are given in hexadecimal. The contents of the TLB, the page table for the first 32 pages, and the cache are as

follows:

TLB			
Index	Tag	PPN	Valid
0	09	4	1
	12	2	1
	10	0	1
	08	5	1
	05	7	1
	13	1	0
	10	3	0
	18	3	0
1	04	1	0
	0C	1	0
	12	0	0
	08	1	0
	06	7	0
	03	1	0
	07	5	0
	02	2	0

Page Table					
VPN	PPN	Valid	VPN	PPN	Valid
00	6	1	10	0	1
01	5	0	11	5	0
02	3	1	12	2	1
03	4	1	13	4	0
04	2	0	14	6	0
05	7	1	15	2	0
06	1	0	16	4	0
07	3	0	17	6	0
08	5	1	18	1	1
09	4	0	19	2	0
0A	3	0	1A	5	0
0B	2	0	1B	7	0
0C	5	0	1C	6	0
0D	6	0	1D	2	0
0E	1	1	1E	3	0
0F	0	0	1F	1	0

2-way Set Associative Cache											
Index	Tag	Valid	Byte0	1	2	3	Tag	Valid	Byte0	1	2	3
0	19	1	99	11	23	11	00	0	99	11	23	11
1	15	0	4F	22	EC	11	2F	1	55	59	0B	41
2	1B	1	00	02	04	08	0B	1	01	03	05	07
3	06	0	84	06	B2	9C	12	0	84	06	B2	9C
4	07	0	43	6D	8F	09	05	0	43	6D	8F	09
5	0D	1	36	32	00	78	1E	1	A1	B2	C4	DE
6	11	0	A2	37	68	31	00	1	BB	77	33	00
7	16	1	11	C2	11	33	1E	1	00	C0	0F	00

Part I (9 points)

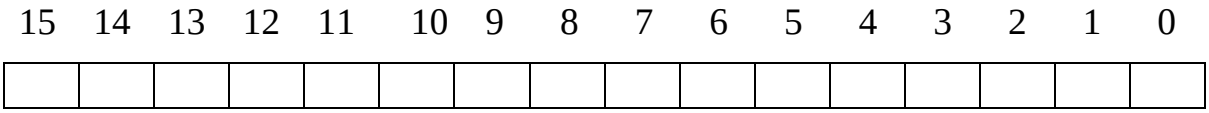
A. The box below shows the format of a virtual address. Indicate (by labeling the diagram) the fields (if they exist) that would be used to determine the following: (If a field doesn't exist, don't draw it on the diagram.)

VPO The virtual page offset

VPN The virtual page number

TLBI The TLB index

TLBT The TLB tag



B. The box below shows the format of a physical address. Indicate (by labeling the diagram) the fields that would be used to determine the following:

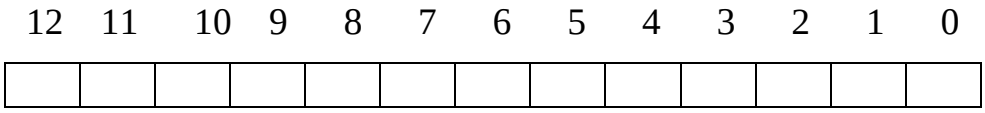
PPO The physical page offset

PPN The physical page number

CO The block offset within the cache line

CI The cache index

CT The cache tag



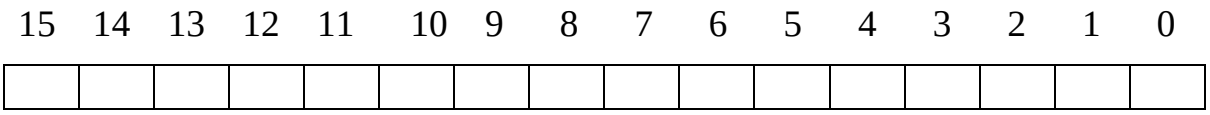
Part II (13 points)

For the given virtual address, indicate the TLB entry accessed, the physical address, and the cache byte value returned in hex. Indicate whether the TLB misses, whether a page fault occurs, and whether a cache miss occurs. If there is a cache miss, enter “-” for “Cache Byte returned”. If there is a page fault, enter “-” for “PPN” and leave parts C and D blank.

Virtual address: *1DDE*

（装订线内不要答题）

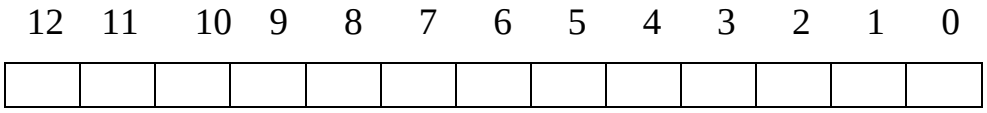
A. Virtual address format (one bit per box)



B. Address translation

Parameter	Value
VPN	0x
TLB Index	0x
TLB Tag	0x
TLB Hit?(Y/N)	
Page Fault?(Y/N)	
PPN	0x

C. Physical address format (one bit per box)



D. Physical memory reference

Parameter	Value
Byte offset	0x
Cache Index	0x
Cache Tag	0x
Cache Hit?(Y/N)	
Cache Byte returned	0x